

# SIMATS ENGINEERING

## ASSESSMENT TOOL 2: Error Analysis Task

**COURSE NAME:** Cloud Computing and  
Big Data Analytics

**COURSE CODE:** CSA15

---

### COURSE OUTCOME COVERED

**CO5:** *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*  
(BL5)

Dave's Taxonomy: Articulation (Level 4)  
Question Count: 5

**Total Marks: 25**

---

### Code of Conduct

I JASMITHA R(192524295)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: JASMITHA R

### Assessment Tasks

#### Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

#### Error Analysis Task Rubric

| Criteria                | Weightage | Level 4 - Exemplary                                 | Level 3 - Proficient         | Level 2 - Developing       | Level 1 - Beginning     |
|-------------------------|-----------|-----------------------------------------------------|------------------------------|----------------------------|-------------------------|
| Identification of Error | 30%       | Accurately identifies the root technical issue      | Identifies most of the issue | Partially identifies issue | Fails to identify issue |
| Cause Analysis          | 20%       | Explains root cause with strong technical reasoning | Reasonable cause analysis    | Basic cause analysis       | Weak analysis           |

|                                      |     |                                             |                                |                     |                         |
|--------------------------------------|-----|---------------------------------------------|--------------------------------|---------------------|-------------------------|
| Corrective Action                    | 25% | Proposes effective and feasible corrections | Reasonable corrections         | Basic corrections   | Ineffective corrections |
| Use of Hadoop / Integration Concepts | 15% | Applies relevant concepts appropriately     | Mostly appropriate application | Partial application | Incorrect application   |
| Clarity                              | 10% | Clear and direct explanation                | Generally clear                | Somewhat unclear    | Poorly presented        |

## Task 1: Uneven Input Splits Causing Idle Reducers in a Hadoop Job

**Scenario:** A Hadoop MapReduce job repeatedly fails or stalls because input splits are uneven in size, and reducers remain idle while waiting on a small number of overloaded mappers or reducers.

### Background and Technical Explanation

In a healthy MapReduce job, work is divided into input splits that are processed in parallel by mapper tasks, and the resulting intermediate key-value pairs are partitioned across reducer tasks so that all reducers finish at roughly the same time. When splits or partitions are unbalanced, the overall job runtime is dictated by the slowest task rather than the average task, which is often called the straggler problem. This directly causes idle reducer slots, wasted cluster capacity, missed SLAs, and in extreme cases job failures due to timeouts or memory exhaustion on the overloaded tasks.

### Root Cause / Design Error

- **Data skew:** The input data or the intermediate key distribution is skewed, meaning a few keys carry a disproportionate share of records, so a handful of map or reduce tasks process far more data than the rest of the cluster.
- **Poor split sizing:** The default InputFormat and block size based splitting was used without accounting for variable record sizes, so some splits contain far more actual work than others even though they appear similar in byte size.
- **Naive hash partitioning:** The default HashPartitioner sends disproportionate volumes of intermediate data to specific reducers when key cardinality is low or unevenly distributed, so most reducers finish quickly and sit idle while one or two remain heavily backlogged.
- **Non splittable input formats:** Compressed files such as plain gzip, or other formats that cannot be split, force an entire file into a single map task, creating oversized and unbalanced splits that cannot be parallelized further.
- **Insufficient combiner usage:** Without a combiner function, large volumes of intermediate data are shuffled across the network to a small

set of reducers, amplifying the imbalance that skewed keys already create.

- **Lack of speculative execution:** Speculative execution is disabled or misconfigured, so a single slow straggler task is allowed to hold up the entire job rather than being duplicated and raced on another node.

## Corrective Actions and Recommendations

- **Custom partitioning:** Implement a custom Partitioner, or apply salting and bucketing of hot keys, so intermediate data is spread evenly across reducers instead of relying purely on a hash of the key.
- **Key salting for hot keys:** For heavily skewed keys, append a random suffix, or salt, to split a single hot key's records across multiple reducers, then perform a secondary aggregation step to combine the partial results.
- **Right size the splits:** Tune `mapreduce.input.fileinputformat.split.minsize` and `split.maxsize`, or use `CombineFileInputFormat`, so multiple small files are combined into balanced splits and large files are broken into proportionate chunks.
- **Use splittable compression:** Switch to splittable compression formats such as bzip2, or Snappy and LZO with indexing, instead of plain gzip, so large compressed files can still be divided across multiple mappers.
- **Enable combiners:** Add a Combiner function to pre aggregate map output locally on each node, reducing the volume of skewed data that must be shipped across the network to any single reducer.
- **Enable speculative execution:** Turn on speculative execution for both map and reduce phases so that unusually slow tasks are automatically duplicated on other nodes, and the first one to finish is used.
- **Monitor and rebalance continuously:** Use the ResourceManager or JobHistory UI and task level counters to identify skewed tasks early, and re-tune partitioning logic as data distribution changes over time.

## Expected Outcome

Balancing split sizes and intermediate key distribution eliminates idle reducers, shortens overall job runtime, and prevents the timeouts and failures

caused by a small number of overloaded tasks holding up the entire job. In production, this typically improves cluster utilization significantly and makes job completion times far more predictable.

## **Task 2: Frequent Data Unavailability After a Single Node Failure in HDFS**

**Scenario:** An HDFS deployment experiences frequent data unavailability whenever a single DataNode fails, even though HDFS is designed to tolerate node failures through replication.

### **Background and Technical Explanation**

HDFS is built around the assumption that hardware failure is the normal case rather than the exception, and it relies on block replication and a centralized namespace managed by the NameNode to survive individual node failures without data loss or downtime. When a properly configured cluster loses a single DataNode, the NameNode should simply route read and write requests to the remaining healthy replicas of each affected block while background re-replication restores the desired replication factor. Frequent unavailability after a single node failure indicates that one or more of these protective mechanisms is missing, misconfigured, or undersized.

### **Root Cause / Design Error**

- **Replication factor set too low:** The `dfs.replication` value is configured at 1 or 2, meaning a single node failure can remove the only, or last remaining, copy of certain blocks before re-replication has a chance to complete.
- **Poor rack awareness and placement policy:** Rack awareness is not configured, so replicas may be placed on nodes that share the same underlying hardware, rack, switch, or power domain, allowing a single failure to take out multiple replicas of the same block simultaneously.
- **NameNode as an unmitigated single point of failure:** The cluster runs a single NameNode with no High Availability standby, no shared edit log through JournalNodes and the QuorumJournalManager, and no automatic failover through a ZooKeeper Failover Controller, so if the failed node is or affects the NameNode, the entire filesystem namespace becomes unavailable.

- **Slow block re-replication:** Default heartbeat and block report intervals, along with under-replicated block detection thresholds, are too slow, leaving the cluster in a vulnerable window after a node fails and before replicas are restored elsewhere.
- **Insufficient DataNode capacity or headroom:** Too few DataNodes, or too little free space across the cluster, prevents the NameNode from finding suitable target nodes to re-replicate under-replicated blocks promptly after a failure.

## Corrective Actions and Recommendations

- **Increase the replication factor:** Set `dfs.replication` to at least 3, which is the standard production default, and use a higher factor for especially critical datasets.
- **Configure rack awareness:** Implement a rack topology script so the default block placement policy distributes replicas across different racks, ensuring that a single rack or power failure cannot take out every copy of a block at once.
- **Deploy NameNode High Availability:** Configure an Active and Standby NameNode pair with QuorumJournalManager and JournalNodes for a shared edit log, along with ZooKeeper based automatic failover, to remove the NameNode as a single point of failure.
- **Tune re-replication responsiveness:** Adjust heartbeat intervals, block report intervals, and the priority given to under-replicated blocks so the NameNode reacts quickly and restores full replication soon after a node is lost.
- **Plan capacity and monitor proactively:** Maintain sufficient spare DataNode capacity across the cluster, and continuously monitor under-replicated and missing block counts using the NameNode web interface or the `hdfs fsck` command so issues are caught before they cause an outage.

## Expected Outcome

With adequate replication, rack aware placement, and NameNode High Availability in place, a single DataNode failure, and even a NameNode

failure, is absorbed transparently by the cluster, and data remains continuously available to clients without manual intervention.

### **Task 3: Duplicate Records from an ETL and Apache NiFi Ingestion Workflow**

**Scenario:** A data ingestion workflow that uses ETL processes together with Apache NiFi produces duplicate records in the downstream analytics store, inflating counts and skewing analytical results.

#### **Background and Technical Explanation**

Apache NiFi and most distributed ingestion frameworks are designed around at least once delivery guarantees rather than exactly once guarantees, because exactly once delivery across an entire distributed pipeline is difficult and expensive to guarantee end to end. This means that under normal operating conditions, and especially after any failure, restart, or network interruption, the same record can legitimately be delivered or processed more than once. If the destination analytics store simply appends every record it receives without any deduplication or idempotency logic, duplicates will accumulate over time and become more noticeable as failures, retries, and restarts occur.

#### **Root Cause / Design Error**

- **At least once delivery without deduplication:** NiFi flows guarantee at least once delivery, and if no deduplication logic exists downstream, retried or replayed FlowFiles are written to the destination more than once.
- **Reprocessing after failure without idempotent writes:** When a processor or the NiFi node restarts after a failure, in flight FlowFiles sitting in queues can be reprocessed, and if the destination such as HDFS, a relational database, or a data warehouse does not enforce a unique key or use upsert semantics, the same record is inserted again as a brand new row.
- **Lack of unique record identifiers:** Source records lack a natural or generated primary key or unique identifier, so the pipeline has no reliable way to detect that an incoming record has already been ingested previously.

- **Multiple concurrent flow paths or duplicated processors:** Misconfigured NiFi flow branching, such as a Funnel or connection accidentally routing the same FlowFile down two separate paths, or duplicated ETL jobs running concurrently over overlapping data windows, causes the same event to be loaded twice.
- **No transaction boundary between extraction and load:** The ETL job re-reads from the same offset or time window after a partial failure, for example a Kafka or NiFi consumer that is not committing offsets correctly, causing the same batch of records to be re-extracted and reloaded from scratch.

## Corrective Actions and Recommendations

- **Enforce idempotent writes at the destination:** Use upsert or merge semantics, such as a MERGE INTO statement, deterministic file naming for PutHDFS, or database level UPSERT operations, so that re-delivery of the same record does not create a new row.
- **Add deduplication keys:** Generate or use a natural unique identifier, such as a record ID, a composite business key, or a content hash, and apply a NiFi DetectDuplicate processor backed by a distributed cache before the final load step.
- **Audit and fix flow topology:** Review the entire NiFi flow for accidental duplicate connections or branches and consolidate them so that each FlowFile has exactly one path to the destination for a given business event.
- **Use exactly once or transactional connectors where available:** Where source systems support it, such as Kafka with transactional producers and consumers, enable exactly once semantics and ensure that offset commits happen only after the corresponding downstream write has succeeded.
- **Maintain checkpoint and windowing discipline:** Ensure ETL extraction jobs commit read offsets or watermarks only after a successful and verified load, so that failures cause safe, bounded re-reads rather than blind reprocessing of already loaded data.
- **Add post load reconciliation as a safety net:** Schedule periodic deduplication or reconciliation jobs, such as scheduled SQL based



deduplication or NiFi QueryRecord checks, to catch and remove any duplicates that slip through the primary controls.

## **Expected Outcome**

Combining idempotent and upsert style writes at the destination with proper deduplication logic and a cleaner flow topology in NiFi removes duplicate records from the analytics store while still preserving the pipeline's fault tolerance and its ability to recover safely from failures.

## **Task 4: Resource Starvation in a YARN Cluster Under Concurrent Job Submissions**

**Scenario:** A YARN cluster suffers from resource starvation, meaning some users' or jobs' tasks wait indefinitely or for excessively long periods, when multiple users submit jobs to the cluster at the same time.

## **Background and Technical Explanation**

YARN, the resource management layer of Hadoop, is responsible for arbitrating cluster memory and CPU across every application running at a given time. The scheduler component decides which application receives the next available container. If the scheduler and its queue configuration do not actively enforce fairness or guaranteed minimum capacity per user or team, a cluster can appear healthy overall, with high aggregate utilization, while individual users or lower priority jobs are effectively starved of resources for long periods, sometimes indefinitely, whenever heavier users are also active.

## **Root Cause / Design Error**

- **FIFO or poorly configured scheduler:** The cluster is using the default FIFO scheduler, or a Capacity or Fair Scheduler configured with only a single default queue, so large or long running jobs submitted first monopolize cluster resources and later jobs must wait for containers to become free.
- **No queue level resource guarantees:** Without defined capacity queues in the Capacity Scheduler, or fair share pools in the Fair Scheduler, for each user, team, or workload type, there is no minimum resource guarantee, which allows aggressive users or jobs to consume the entire cluster.

- **No preemption configured:** Even where queues or pools exist, preemption is disabled, so a queue that is currently over its allocated capacity cannot be forced to release containers back to an under-served queue, prolonging the starvation of the affected users.
- **Unbounded resource requests:** No per-user or per-queue maximum resource limits are set, so a single user's job can request nearly all cluster memory and vCores, starving every other job submitted around the same time.
- **Container sizing mismatch:** Overly large container size requests for memory or vCores relative to node capacity cause resource fragmentation, where enough aggregate resources exist across the cluster but not enough is free on any single node to satisfy a pending request.

## Corrective Actions and Recommendations

- **Switch to the Capacity or Fair Scheduler:** Replace FIFO scheduling with the Capacity Scheduler, which offers hierarchical queues with guaranteed capacity per team or workload, or the Fair Scheduler, which provides equal or weighted fair sharing across all currently running jobs.
- **Define queues with minimum and maximum capacity:** Configure per-user or per-team queues with a guaranteed minimum capacity and an enforced maximum capacity cap, so that no single job or user is able to exhaust the entire cluster.
- **Enable preemption:** Turn on preemption so that queues running below their guaranteed capacity can reclaim resources from queues that are currently over their allocated capacity, within a configurable grace period.
- **Set sensible container and resource limits:** Right-size `yarn.scheduler.minimum-allocation-mb` and `maximum-allocation-mb`, as well as the corresponding vCore settings, and apply per-user or per-queue resource caps to prevent any single job from monopolizing the cluster.
- **Prioritize and label workloads:** Use YARN node labels and application priorities to separate latency sensitive or high priority

workloads from batch or best effort jobs, and consider dedicated queues for interactive workloads versus long running batch workloads.

- **Monitor and tune continuously:** Track queue utilization, the number of pending versus running containers, and application wait times through the ResourceManager user interface and metrics, and adjust queue capacities regularly as usage patterns evolve over time.

## **Expected Outcome**

A properly configured Capacity or Fair Scheduler, combined with defined queues, resource caps, and preemption, ensures fair and predictable resource sharing across concurrent users and jobs, and effectively eliminates resource starvation on the cluster.

## **Task 5: Backup Strategy Restores Outdated Data in Distributed Storage**

**Scenario:** A backup strategy for a distributed storage system restores outdated, or stale, data after a recovery event, rather than restoring the most recent consistent state of the data.

## **Background and Technical Explanation**

A backup and recovery strategy for distributed storage is only as good as its recovery point objective, which is the maximum acceptable amount of data loss measured in time, and its recovery time objective, which is how quickly service can be restored after a failure. When a restore operation returns data that is noticeably older than the point of failure, it usually means the backup design only captures periodic full snapshots without any mechanism to recover the changes that happened after the last snapshot, or that the replication used for backup purposes is asynchronous and was allowed to lag too far behind the primary system.

## **Root Cause / Design Error**

- **Infrequent or lagging backup and snapshot schedule:** Backups or HDFS snapshots are taken too infrequently relative to the rate of data change, so the most recent available backup point is significantly behind the actual state of the data at the time of failure.
- **Asynchronous replication used as the sole backup mechanism:** Reliance on asynchronous replication, such as cross cluster DistCp jobs

or asynchronous replica synchronization, means the backup or secondary copy can lag behind the primary system, and a failure occurring right after a write can result in restoring a state from before that write.

- **No write ahead or edit log replay before restore:** The recovery process restores only the last full snapshot without replaying the NameNode edit logs, or an equivalent write ahead log, generated after that snapshot, which discards recently committed changes that were never captured in the snapshot itself.
- **Backup validation and versioning gaps:** There is no verification that a given backup is complete, internally consistent, and correctly timestamped before it is relied upon, so a partially completed or corrupted backup cycle can silently become the latest restore point without anyone noticing.
- **Lack of point in time recovery capability:** The backup design only supports restoring to fixed full backup intervals rather than to a specific point in time, so any change made between two backup windows is unrecoverable purely by design, regardless of how the failure occurred.

## **Corrective Actions and Recommendations**

- **Increase backup and snapshot frequency:** Align snapshot and backup intervals with the actual rate of data change and with a clearly defined recovery point objective, so that the maximum acceptable data loss window is well understood and deliberately minimized.
- **Combine full snapshots with incremental logs:** Pair periodic full snapshots with continuous incremental backups or edit log shipping, and replay those logs during recovery to bring the restored copy forward to the actual point of failure.
- **Use synchronous or near real time replication for critical data:** For data with a low tolerance for staleness, use synchronous replication or very frequent incremental synchronization rather than relying solely on asynchronous batch style replication.
- **Validate and version every backup:** Automate integrity checks, such as checksums and consistency validation, after every backup or snapshot job, and retain versioned and timestamped backups so that a

corrupted or incomplete backup is never mistakenly selected as the restore point.

- **Implement point in time recovery:** Design the backup and restore process so it can replay write ahead or edit logs up to a specific timestamp, enabling recovery to the most recent consistent state of the system rather than only to the last full backup boundary.
- **Regularly test disaster recovery procedures:** Periodically perform full restore drills to confirm that the actual recovery point achieved in practice matches the defined recovery point objective, and adjust the backup frequency or overall architecture whenever gaps are discovered.

### **Expected Outcome**

Replacing an infrequent, snapshot only backup approach with a combination of frequent snapshots, continuous log shipping, validated versioning, and point in time recovery ensures that restores reflect the most current consistent state of the system rather than stale, outdated data.